CPSC 250 - Programming for Data Manipulation
Final Exam – Solution Set

Part I: Multiple Choice (30 points)

Correct Answers:

1. **B** – Lists are mutable, and b and a point to the same object.
2. **B** – dropna() removes missing data rows or columns.
3. **B** – ___str___ defines the string representation.
4. **C** – Inheritance promotes code reuse.
5. **C** – The base case defines when recursion stops.
6. **D** – Lists are mutable; tuples are immutable.
7. **B** – curve_fit finds parameters that best fit a function.
8. **C** – Balanced BSTs allow O(log n) searches.
9. **A** – {} defines an empty dictionary.
10. **A** – ___eq___ customizes == behavior.
11. **D** – read_csv() returns a DataFrame.
12. **C** – Operator overloading allows operators to work with user types.
13. **D** – .mean() returns the average.
14. **B** – Polymorphism means shared interface, different behavior.
15. **C** – Inheritance uses syntax class A(B):

Part II: Error Identification (15 points)

Original Code:

```
def reverse_list(lst):
    if len(lst) = 0:
        return []
    else:
        return reverse_list(lst[1:]) + lst[0]
```

Corrected Code:

```
def reverse_list(lst):
    if len(lst) == 0:
        return []
    else:
        return reverse_list(lst[1:]) + [lst[0]]
```

Explanation:

- The condition should use ==, not =.
- You must concatenate lists, not a list and a single value. So use [lst[0]].

Part III: Code Writing (30 points)

Q1. Palindrome Checker (6 points)

```
def is_palindrome(s):
    return s == s[::-1]
```

Q2. Student class with ___str___ and ___lt___ (8 points)

```
class Student:
    def __init__(self, name, gpa):
        self.name = name
        self.gpa = gpa

    def __str__(self):
        return f"{self.name}, GPA: {self.gpa}"

    def __lt__(self, other):
        return self.gpa < other.gpa
```

Q3. Load and filter DataFrame (8 points)

```
import pandas as pd

def load_and_filter(filename):
    df = pd.read_csv(filename)
    return df[df['score'] > 80]
```

Q4. Animal classes with polymorphism (8 points)

```
class Animal:
    def make_sound(self):
        return "..."

class Dog(Animal):
    def make_sound(self):
        return "woof"

class Cat(Animal):
    def make_sound(self):
        return "meow"

# Demonstrate polymorphism
animals = [Dog(), Cat(), Dog()]
```

```
for animal in animals:
    print(animal.make_sound())
```

Part IV: Code Comprehension (25 points)

Original Code:

```
import numpy as np
from scipy.optimize import curve_fit

def model(x, a, b):
    return a * np.exp(b * x)

def fit_data(x, y):
    popt, pcov = curve_fit(model, x, y)
    print("a =", popt[0])
    print("b =", popt[1])
```

Line-by-line Comments:

- import numpy as np Load NumPy for numeric operations.
- from scipy.optimize import curvefit Import curve fitting function.
- def model(x, a, b): Define a model function with two parameters.
- return a * np.exp(b * x) Return exponential function output.
- def fitdata(x, y): Define function that fits data to the model.
- popt, pcov = curvefit(...) Fit model to data, returning parameters and covariances.
- print("a =", popt[0]) Print the best-fit value for a.
- print("b =", popt[1]) Print the best-fit value for b.

What do popt **and** pcov **represent?**

- popt: A list of optimal values for the parameters (a, b).
- pcov: The estimated covariance matrix of the parameter estimates.